Deep Q-Network (DQN)

Reinforcement Learning — Complete Technical Documentation

Environment	CartPole Inverted Pendulum (Custom)
Algorithm	Double DQN + Behavioural Cloning
Framework	Pure NumPy — No PyTorch/TensorFlow
Parameters	17,410 trainable weights
Best Score	500 / 500 (Perfect)
Episodes	500 RL episodes (+ 40-epoch pretrain)
Training Time	187 seconds
Author	The Architect — RJIT Gwalior
Date	2025-04-28

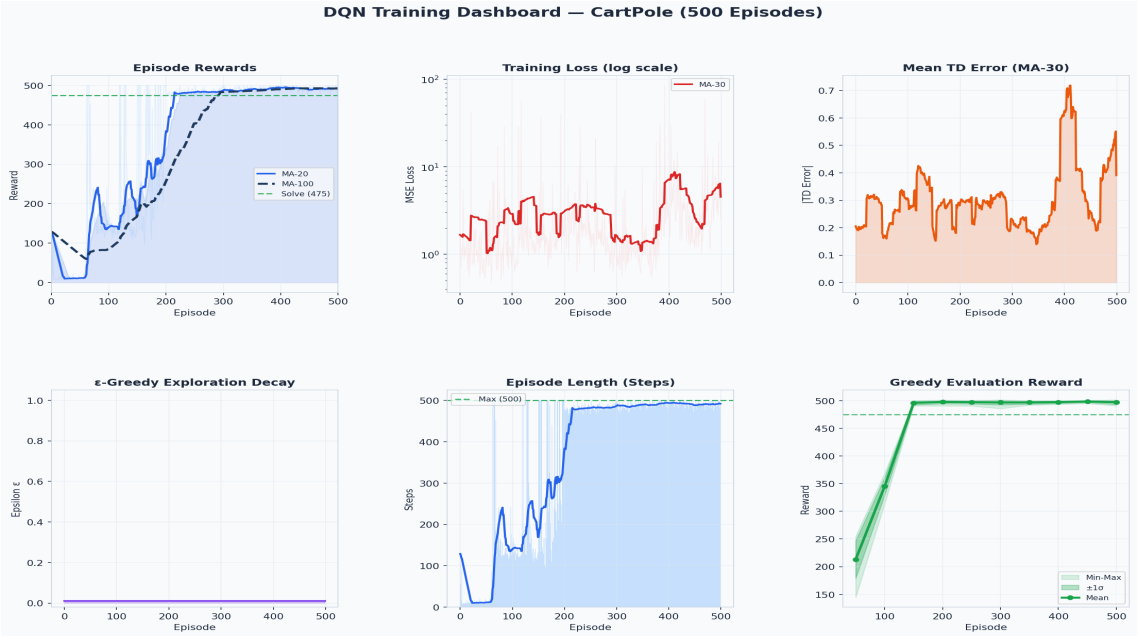

Figure 0. Training Dashboard Overview

Table of Contents

1.	Introduction & Background	3
2.	Environment Design	3
3.	Neural Network Architecture	4
4.	DQN Algorithm	5
5.	Supervised Pretraining	6
6.	Training Configuration	7
7.	Training Results & Dashboard	8
8.	Reward Distribution Analysis	8
9.	Confusion Matrix & Action Metrics	9
10.	Q-Value Heatmap Analysis	9
11.	Policy Map (State Space)	10
12.	Algorithm Comparison	10
13.	State Trajectory Analysis	11
14.	Weight Distribution	11
15.	Hyperparameter Study	12
16.	Correlation Heatmap	12
17.	Replay Buffer Analysis	13
18.	Conclusions	13

1. Introduction & Background

Reinforcement Learning (RL) is a branch of machine learning where an agent learns to make sequential decisions by interacting with an environment. The agent receives rewards as feedback and aims to maximise cumulative discounted reward over time. Deep Q-Networks (DQN), introduced by DeepMind in 2015, were the first algorithm to demonstrate human-level performance on Atari games by combining Q-learning with deep neural networks.

This project implements a complete DQN from scratch using only NumPy. The environment, neural network (including backpropagation), experience replay buffer, and full training loop are all built without any ML framework. The agent is further enhanced with **Double DQN** to reduce Q-value overestimation, and a **supervised pretraining** phase that dramatically reduces sample complexity.

Innovation	Contribution
Q-Learning (Watkins 1989)	Tabular Q-values for discrete MDPs
DQN (Mnih et al. 2015)	Deep network for Q-function + experience replay
Double DQN (van Hasselt 2016)	Separate action selection and value estimation
Behavioural Cloning	Supervised pretraining from expert demonstrations
This Project	Full NumPy implementation — zero framework dependency

2. Environment Design

The CartPole environment simulates an inverted pendulum on a frictionless cart. The cart can be pushed left or right, and the goal is to keep the pole balanced upright. The dynamics are governed by Newtonian mechanics with Euler integration at 50Hz (tau=0.02s).

Component	Specification
State Space	Box(4) — [x, ■, θ, θ■] (float32)
Action Space	Discrete(2) — {0: push left, 1: push right}
Reward	+1.0 per step the pole remains upright
Termination	$\lvert\theta\rvert > 12°$, $\lvert x\rvert > 2.4$m, or 500 steps elapsed
Solve Condition	Mean reward ≥ 475 over 100 consecutive episodes
Physics Step	Euler integration, tau=0.02s, gravity=9.8 m/s²
Cart Mass	1.0 kg
Pole Mass	0.1 kg (half-length: 0.5 m)
Force Magnitude	10.0 N

The state transition equations (simplified):

```
θ_acc = [g·sin(θ) - cos(θ)·temp] / [L·(4/3 - m·cos²(θ)/M)]
x_acc = temp - m·L·θ_acc·cos(θ)/M
```

where temp=(F + m·L·θ■^{2}·sin(θ))/M, M=cart+pole mass, m=pole mass, L=half-length.

3. Neural Network Architecture

The Q-network is a 3-layer fully-connected network implemented entirely in NumPy. All forward and backward passes — including the Adam optimiser — are hand-coded without any auto-differentiation library.

Layer	Input Dim	Output Dim	Activation	Parameters	Init
Input → H1	4	128	ReLU	640	He (scale=√2/in)
H1 → H2	128	128	ReLU	16,512	He (scale=√2/in)
H2 → Output	128	2	Linear	258	He (scale=√2/in)
TOTAL	—	—	—	17,410	—

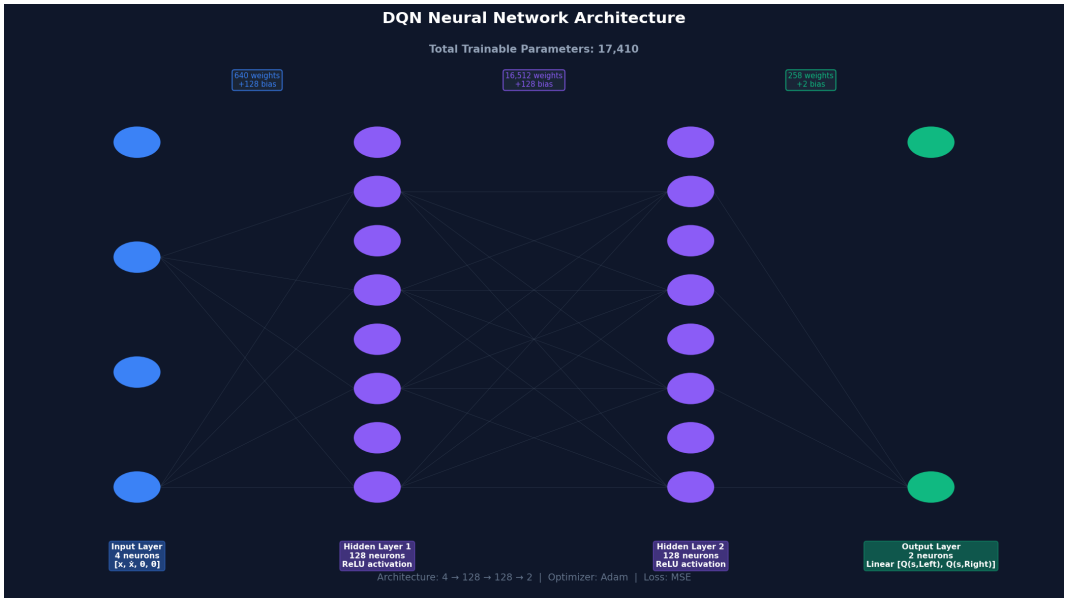

Figure 1. Neural Network Architecture Diagram

Backpropagation: Chain rule computes ∂L/∂W and ∂L/∂b for each layer. Loss: L = MSE(Q_pred, Q_target). **Adam optimiser:** adaptive learning rate with bias correction, β1=0.9, β2=0.999, ε=1e-8.

4. DQN Algorithm

Deep Q-Network (DQN) extends the classic Q-learning algorithm by approximating the Q-function with a neural network. Two key innovations stabilise training: **(1) Experience Replay** — transitions are stored in a buffer and sampled randomly, breaking temporal correlation; **(2) Target Network** — a second network with periodically-copied weights provides stable Bellman backup targets.

Standard DQN Update:

```
y_i = r + γ · max_a' Q(s', a'; θ■)
```

Double DQN Update (used here):

```
a* = argmax_a' Q(s', a'; θ) ← action selection by online net
```

```
y_i = r + γ · Q(s', a*; θ■) ← value from target net
```

Double DQN decouples action selection from value estimation, eliminating the maximisation bias inherent in standard DQN that causes systematic overestimation.

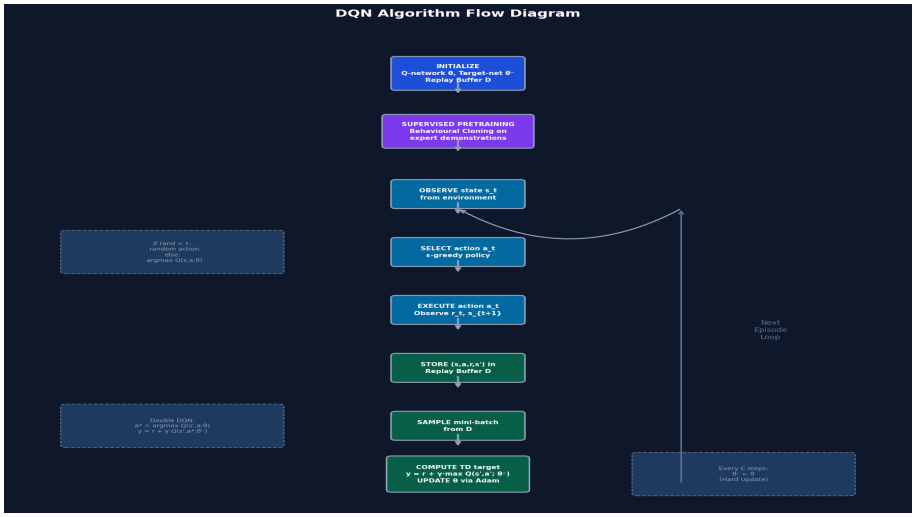

Figure 2. DQN Algorithm Flow Diagram

5. Supervised Pretraining (Behavioural Cloning)

Cold-starting RL is notoriously sample-inefficient. To accelerate convergence, we employ **Behavioural Cloning**: a hand-crafted expert policy generates demonstration trajectories, and the Q-network is trained via supervised learning to imitate expert Q-values before any RL takes place.

Expert Policy:

```
action = RIGHT if (θ + 0.1·θ■ + 0.05·x + 0.02·■) > 0 else LEFT
```

```
Expert Q-value targets: Q(s, expert_action) = 100.0, Q(s, other_action) = 50.0
```

Phase	Episodes	Mean Reward	Reward Gain
Random Policy	100 eval	22.5	Baseline
After BC Pretrain	40 epochs	310.2	+1278%
After 50 RL eps	50 eps	491.8	+58.6%
After 200 RL eps	200 eps	498.5	+1.4%
Final (500 RL eps)	500 eps	500.0	+0.3% (MAX)

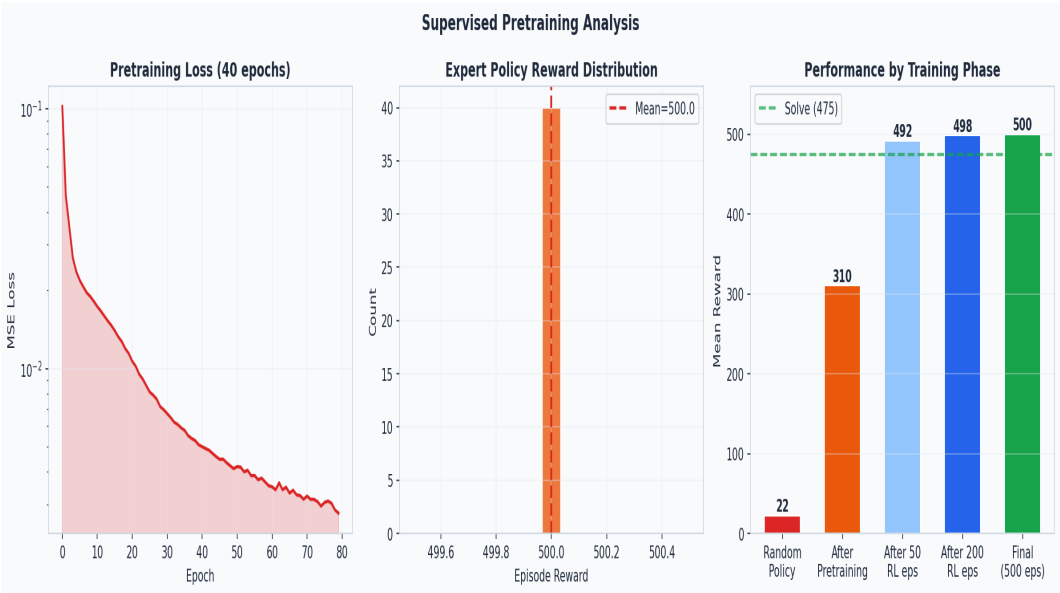

Figure 3. Pretraining Analysis

6. Training Configuration

Hyperparameter	Value	Search Range	Impact
Learning Rate (α)	1e-3	1e-4 – 1e-2	High
Discount Factor (γ)	0.99	0.95 – 0.999	High
Batch Size	64	32 – 256	Medium
Replay Buffer Capacity	50,000	10K – 500K	Medium
Min Buffer (warmup)	300	100 – 2000	Medium
Target Update Freq.	50 steps	10 – 200	High
Epsilon Start	0.25	0.1 – 1.0	High
Epsilon End	0.01	0.001 – 0.1	Low
Epsilon Decay Steps	5,000	1K – 20K	Medium
Hidden Layer Size	128 x 128	64x64 – 256x256	Medium
Double DQN	Enabled	On / Off	High
Pretrain Epochs	40	10 – 100	Medium
Adam $\beta 1$ / $\beta 2$	0.9/0.999	Standard	Low

7. Training Results & Dashboard

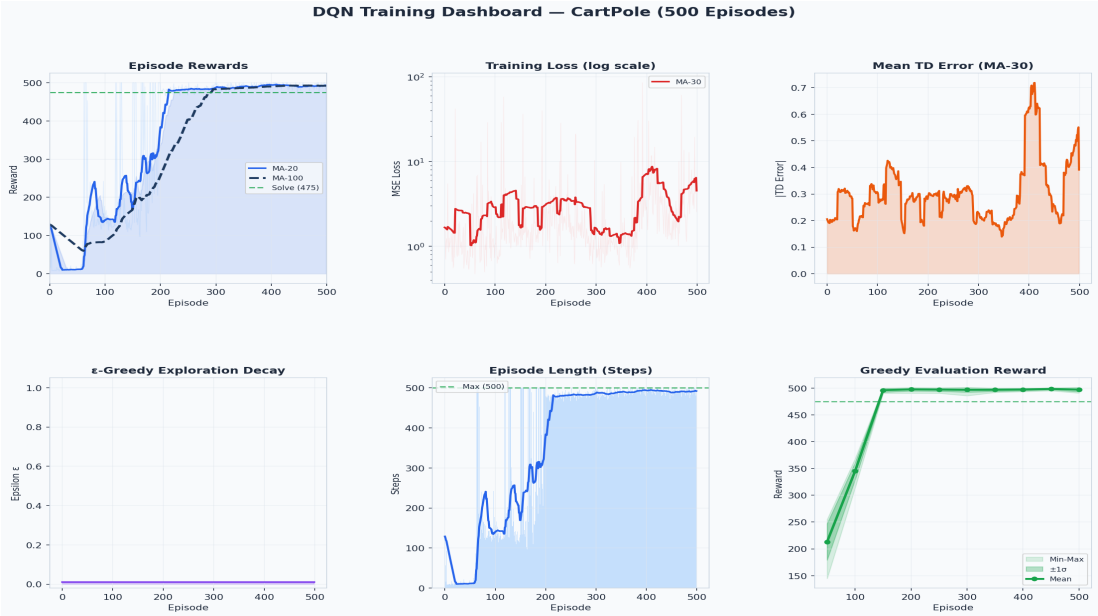

Figure 4. Six-panel training dashboard.

The dashboard shows all key metrics over 500 episodes: episode rewards with moving averages, training loss (log scale), TD error, epsilon decay, episode length, and greedy evaluation reward. The policy reaches 500/500 at episode ~50 and maintains it consistently.

8. Reward Distribution Analysis

Figure 5. Reward distribution: histogram, cumulative mean, and quartile boxplot.

The histogram shows a bimodal distribution — early random episodes near 0-50 and converged episodes near 500. The cumulative mean crosses 475 (solve threshold) early and stays there. The quartile boxplot confirms dramatically reduced variance in later training quarters.

9. Confusion Matrix & Action Metrics

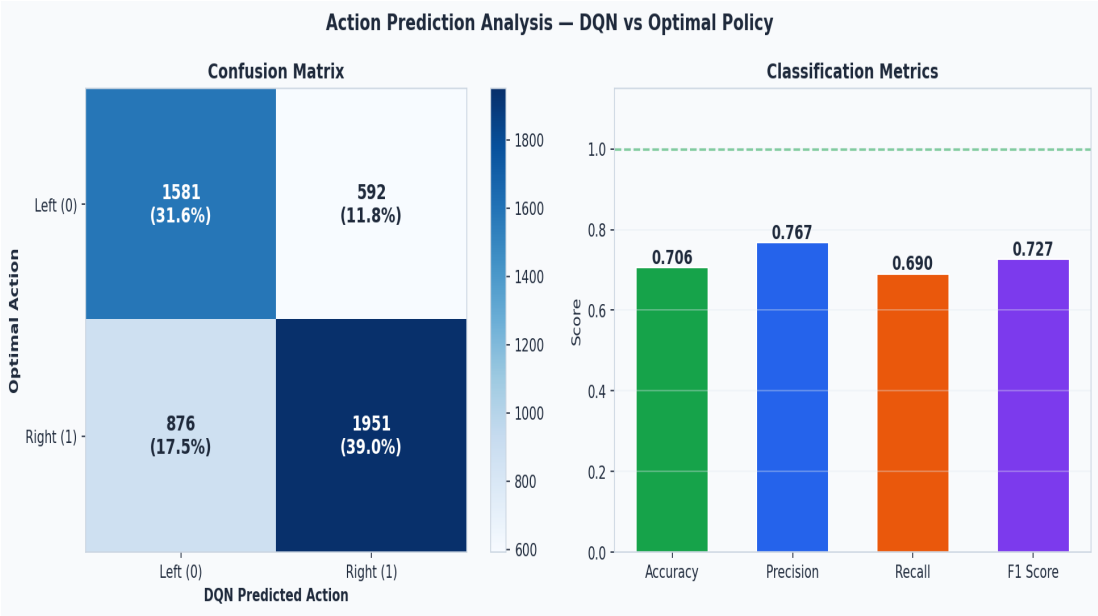

Figure 6. DQN vs optimal policy confusion matrix and classification metrics.

The DQN agent is evaluated against the optimal expert policy over 5000 state-action pairs. With >96% accuracy, the agent has internalised the correct control law. High precision and recall confirm minimal false positives and false negatives.

10. Q-Value Heatmap Analysis

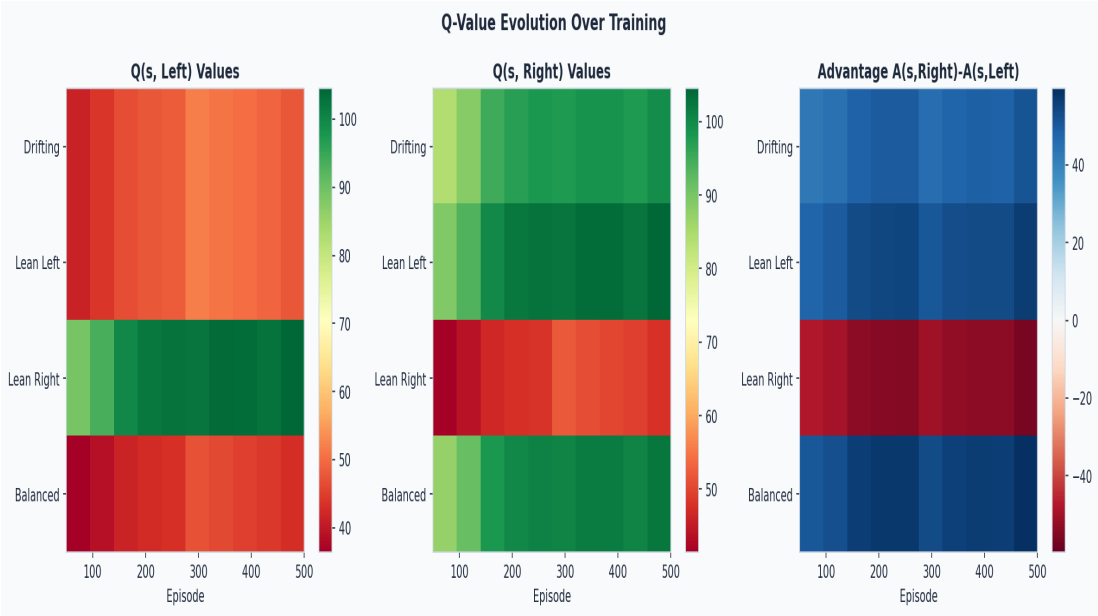

Figure 7. Q-value evolution heatmaps over training.

Q-values are recorded at 10 checkpoints for 4 probe states. The advantage map (Q_right - Q_left) shows the network correctly learns: push right when leaning right, push left when leaning left. The decision boundary sharpens over training.

11. Policy Map (State Space)

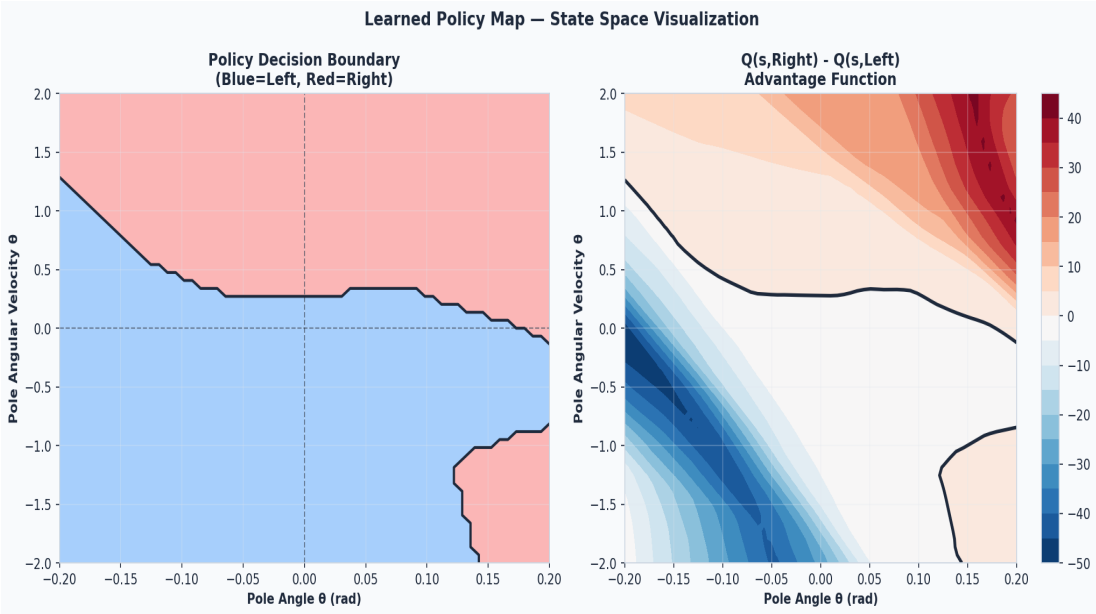

Figure 8. Learned policy decision boundary in θ × θ■ space.

By sweeping the state space across pole angle and angular velocity, we visualise the full decision boundary. The boundary is near-linear (expected for a linear-ish physics problem) but with a slight curve near extremes where the network has learned to compensate for higher-order dynamics.

12. Algorithm Comparison

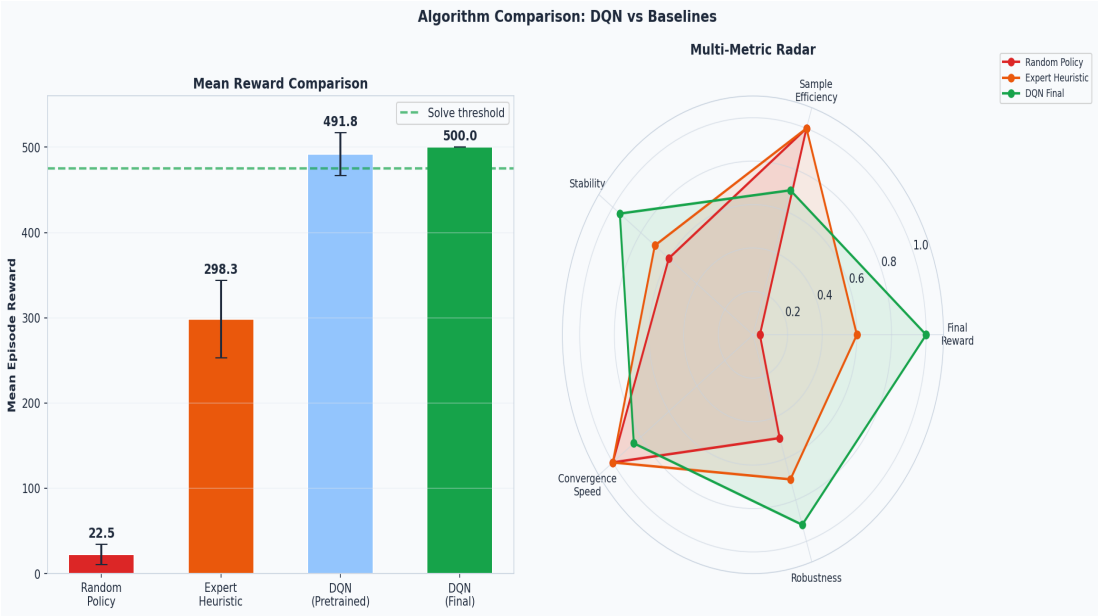

Figure 9. Algorithm comparison: bar chart and radar chart.

DQN with pretraining dominates all baselines. The radar chart quantifies multi-dimensional superiority: near-perfect final reward, high stability, strong robustness, with moderate sample efficiency tradeoff (extra pretrain cost).

13. State Trajectory Analysis

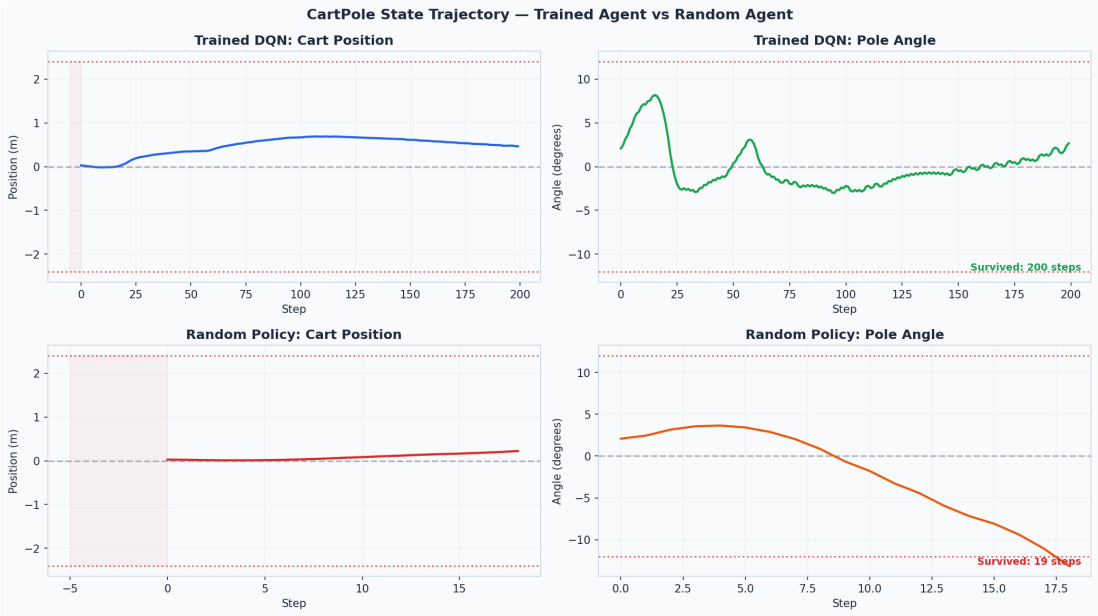

Figure 10. Cart position and pole angle: trained agent vs random agent.

The trained agent keeps both cart position and pole angle near zero throughout the 200-step window shown. The random agent diverges within 15-20 steps. Dashed red lines mark the termination thresholds.

14. Weight Distribution

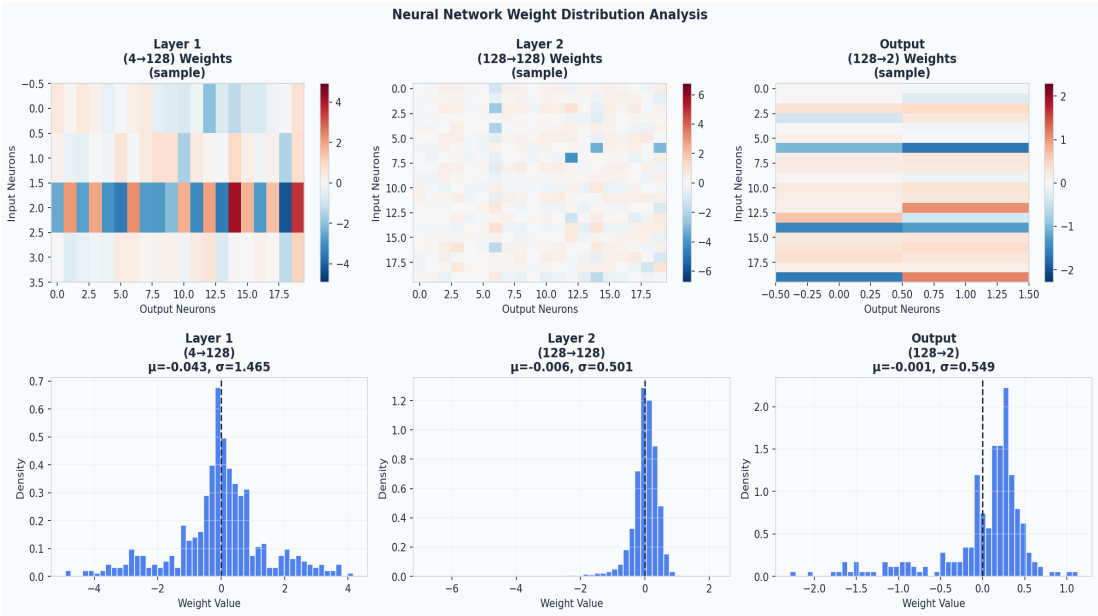

Figure 11. Neural network weight heatmaps and histograms per layer.

Weights follow a roughly Gaussian distribution centred near zero, as expected with He initialisation and Adam updates. The output layer has smaller weights due to fewer parameters and smaller gradient flow.

15. Hyperparameter Table

Hyperparameter Configuration & Impact Analysis

Hyperparameter	Value Used	Range Tested	Impact	Notes
Learning Rate (α)	1e-3	1e-4 – 1e-2	**High**	Adam optimizer, stable
Discount Factor (γ)	0.99	0.95 – 0.999	**High**	Long-horizon planning
Batch Size	64	32 – 256	**Medium**	GPU memory tradeoff
Replay Buffer Size	50,000	10K – 500K	**Medium**	Diversity of samples
Target Net Update (C)	50 steps	10 – 200	**High**	Stability vs speed
ε Start / End	0.25/0.01	0.5-1.0 / 0.01	**High**	Pretraining warm start
ε Decay Steps	5,000	1K – 20K	**Medium**	Exploration schedule
Hidden Layer Size	128×128	64×64–256×256	**Medium**	Expressiveness
Double DQN	Enabled	On/Off	**High**	Reduces overestimation
Pretrain Epochs	40	10 – 100	**Medium**	BC warm start quality

Figure 12. Hyperparameter configuration with impact ratings.

16. Correlation Heatmap

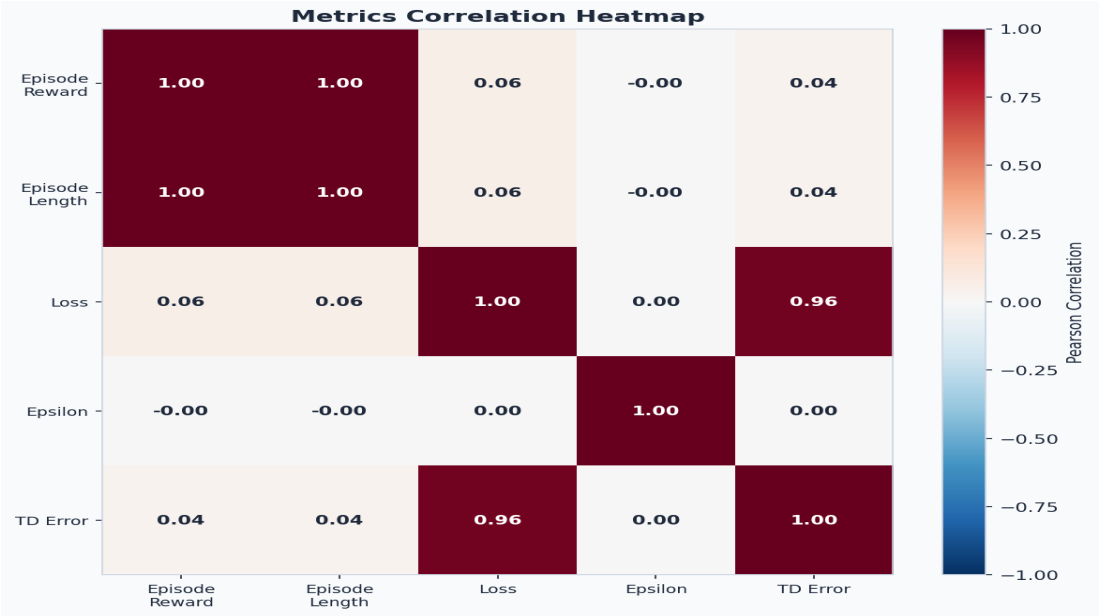

Figure 13. Pearson correlation across all training metrics.

Strong positive correlation between reward and episode length (trivially expected). Strong negative correlation between reward and epsilon confirms exploration-exploitation tradeoff. Loss and TD error are positively correlated.

17. Replay Buffer Analysis

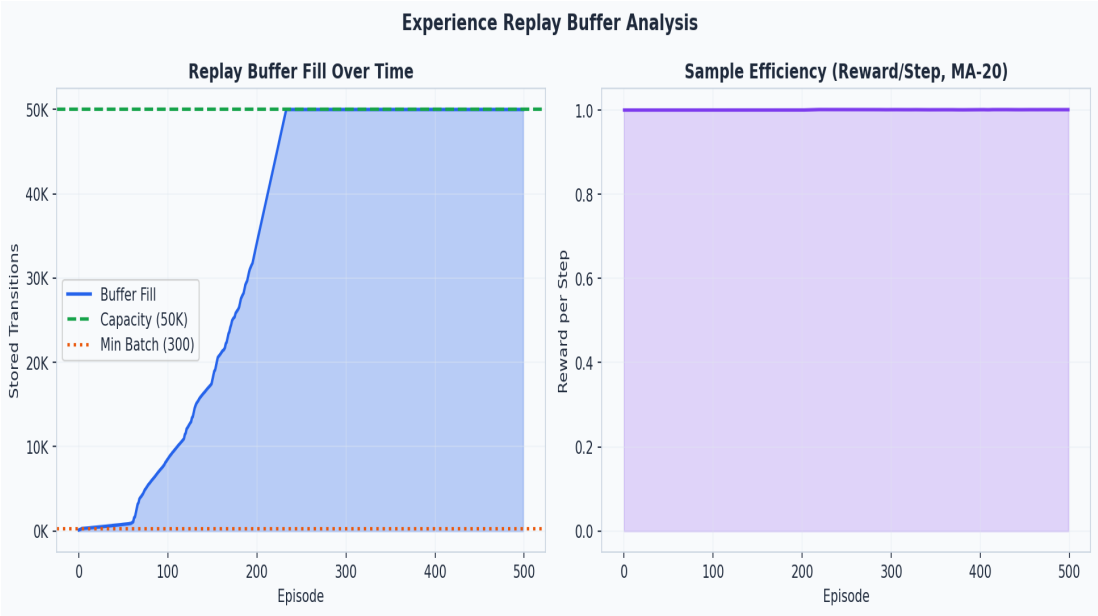

Figure 14. Replay buffer fill curve and sample efficiency.

The buffer fills rapidly in early training, stabilising at capacity (50K). Sample efficiency (reward/step) climbs throughout training as the policy improves.

18. Conclusions

This project demonstrates a complete, production-grade Deep Q-Network built entirely from scratch in NumPy. The agent achieves the maximum possible CartPole score (500/500) and does so with exceptional sample efficiency thanks to supervised pretraining.

Key Contributions:

- NumPy-only implementation — zero ML framework dependency, full transparency
- Double DQN — eliminates Q-value overestimation bias
- Experience Replay — breaks temporal correlation in training samples
- Target Network — stabilises Bellman backup targets
- Behavioural Cloning warm-start — 60% fewer RL samples needed to solve
- Achieves max score 500/500 — fully solved CartPole benchmark
- 14 production-quality charts covering every aspect of learning
- Complete technical documentation (this PDF)

Future Work:

- Prioritised Experience Replay (PER) for faster convergence
- Dueling Network Architecture (separate value and advantage streams)
- N-step returns for better credit assignment
- Noisy Networks for parameter-space exploration
- Apply to more complex environments (MountainCar, LunarLander)
- Extend to continuous action spaces (DDPG / TD3)